Paradigmas de Programação

Curso: Ciência da Computação

Flávia Martins

Centro Universitário Dom Helder

Strings em C

- Em C, uma string é um **array de caracteres** do tipo `char`.
- Ela representa uma sequência de caracteres armazenados em posições adjacentes de memória.
- Toda string válida em C termina com o caractere especial `\0`, chamado de **terminador nulo**.
- Esse terminador indica onde a string acaba.

Exemplo:

```
char nome[6] = "Julio";
```

A memória fica assim:

```
J   u   l   i   o   \0
```

Por que o \0 é tão importante?

- Funções de string em C percorrem o vetor até encontrar \0.
- Sem esse terminador, a função pode continuar lendo memória indevida.
- Por isso, o tamanho do vetor deve considerar:

$$\text{número de caracteres} + 1$$

Exemplos:

```
char a[3] = "Oi";    // correto:  O i \0
char b[6] = "Teste";   // correto:  5 letras + \0
```

Tamanho do vetor x tamanho da string

- **Tamanho do vetor**: quantidade total de posições reservadas.
- **Tamanho da string**: quantidade de caracteres antes do `\0`.

Exemplo:

```
char str[20] = "C";
```

- O vetor possui 20 posições.
- A string possui apenas 1 caractere.
- A função `strlen(str)` retornará 1.

Formas de declarar strings

- Declarar apenas o vetor:

```
1 char str[10];
```

- Declarar e inicializar:

```
1 char str[10] = "Oi";
```

- Deixar o compilador calcular o tamanho:

```
1 char str[] = "Programacao";
```

- Nesse último caso, o compilador reserva espaço suficiente para os caracteres e o `\0`.

Aspas duplas x aspas simples

- `"A"` representa uma **string**.
- `'A'` representa um **caractere**.

Exemplo:

```
1 char letra = 'A';
2 char texto[] = "A";
```

- `letra` guarda um único caractere.
- `texto` guarda dois elementos:

`'A'` `\0`

Acessando caracteres individualmente

Como strings são arrays, podemos acessar cada caractere por índice.

```
1  #include <stdio.h>
2
3  int main() {
4      char str[6] = "Teste";
5
6      printf("%c\n", str[0]); // T
7      printf("%c\n", str[1]); // e
8
9      str[0] = 'L';
10     printf("%s\n", str);    // Leste
11
12     return 0;
13 }
```

Escrevendo strings na tela

Para exibir strings, normalmente usamos `printf()` com o especificador `%s`.

```
1 #include <stdio.h>
2
3 int main() {
4     char nome[] = "Maria";
5     printf("Nome: %s\n", nome);
6     return 0;
7 }
```

- `%s` diz ao `printf` que será impresso um texto terminado em `\0`.

Leitura de strings: cuidado com `gets`

Historicamente, muitos materiais mostram `gets(str)`, mas ela é **insegura**, pois não limita a quantidade de caracteres lidos.

Evite:

```
gets(nome);
```

Prefira:

```
fgets(nome, sizeof(nome), stdin);
```

- `fgets` lê com limite de tamanho.
- Isso ajuda a evitar **estouro de buffer**.

Usando fgets

Protótipo:

```
fgets(str, tamanho, stdin);
```

- `str`: vetor onde a string será armazenada.
- `tamanho`: máximo de caracteres a considerar.
- `stdin`: entrada padrão, normalmente o teclado.

Exemplo:

```
#include <stdio.h>

int main() {
    char nome[30];

    printf("Digite seu nome: ");
    fgets(nome, 30, stdin);

    printf("Voce digitou: %s", nome);
    return 0;
}
```

Observação sobre o `fgets`

- Se houver espaço, o `fgets` também armazena o `\n` digitado ao pressionar Enter.
- Muitas vezes é interessante remover esse `\n`.

Exemplo:

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char nome[30];
6
7      fgets(nome, sizeof(nome), stdin);
8      nome[strcspn(nome, "\n")] = '\0';
9
10     printf("[%s]\n", nome);
11     return 0;
12 }
```

Escrevendo strings com fputs

Outra função útil para saída de strings é `fputs()`.

```
1 #include <stdio.h>
2
3 int main() {
4     char msg[] = "Ola, mundo!";
5     fputs(msg, stdout);
6     fputs("\n", stdout);
7     return 0;
8 }
```

- `stdout` representa a saída padrão.
- Diferente de `printf`, `fputs` não usa máscara como `%s`.

Biblioteca `string.h`

A biblioteca `string.h` fornece funções prontas para manipulação de strings.

Principais funções:

- `strlen` — tamanho da string
- `strcpy` — copiar string
- `strcat` — concatenar strings
- `strcmp` — comparar strings

Inclusão:

```
1 #include <string.h>
```

strlen: tamanho da string

strlen(str) retorna a quantidade de caracteres da string, sem contar o \0.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char palavra[] = "teste";
6     printf("Tamanho: %zu\n", strlen(palavra));
7     return 0;
8 }
```

Saída:

```
1 Tamanho: 5
```

strcpy: copiando strings

Em C, não podemos copiar strings com operador = quando elas são arrays.

Errado:

```
1 char a[20] = "Oi";
2 char b[20];
3 b = a; // erro
```

Certo:

```
1 #include <string.h>
2
3 char a[20] = "Oi";
4 char b[20];
5 strcpy(b, a);
```

Exemplo completo com strcpy

```c
#include <stdio.h>
#include <string.h>

int main() {
    char origem[20] = "Hello World";
    char destino[20];

    strcpy(destino, origem);

    printf("Origem : %s\n", origem);
    printf("Destino: %s\n", destino);
    return 0;
}
```

Cópia manual caractere por caractere

Antes de usar biblioteca, é importante entender como a cópia funciona.

```
#include <stdio.h>

int main() {
    int i = 0;
    char origem[20] = "Hello World";
    char destino[20];

    while (origem[i] != '\0') {
        destino[i] = origem[i];
        i++;
    }
    destino[i] = '\0';

    printf("%s\n", destino);
    return 0;
}
```

strcat: concatenando strings

`strcat(destino, fonte)` adiciona a string `fonte` ao final de `destino`.

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char nome[30] = "Ana";
6      char sobrenome[] = " Silva";
7
8      strcat(nome, sobrenome);
9
10     printf("%s\n", nome);
11     return 0;
12 }
```

Saída:

```
1  Ana Silva
```

Cuidado ao usar `strcat`

- O vetor de destino precisa ter espaço suficiente.
- Caso contrário, haverá sobrescrita de memória.

Exemplo seguro:

```
1 char nome[30] = "Ana";
2 strcat(nome, " Silva");
```

Exemplo perigoso:

```
1 char nome[5] = "Ana";
2 strcat(nome, " Silva");
```

strcmp: comparando strings

`strcmp(str1, str2)` compara duas strings.

- Retorna `0` se forem iguais.
- Retorna valor diferente de zero se forem diferentes.

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char a[] = "casa";
6      char b[] = "casa";
7      char c[] = "carro";
8
9      printf("%d\n", strcmp(a, b)); // 0
10     printf("%d\n", strcmp(a, c)); // diferente de 0
11     return 0;
12 }
```

Exemplo prático de comparação

```c
#include <stdio.h>
#include <string.h>

int main() {
    char senha[20];

    printf("Digite a senha: ");
    fgets(senha, sizeof(senha), stdin);
    senha[strcspn(senha, "\n")] = '\0';

    if (strcmp(senha, "1234") == 0) {
        printf("Senha correta!\n");
    } else {
        printf("Senha incorreta!\n");
    }

    return 0;
}
```

Percorrendo uma string

Como strings são arrays, podemos percorrê-las com laços.

```
1  #include <stdio.h>
2
3  int main() {
4      char texto[] = "Cadeira";
5      int i = 0;
6
7      while (texto[i] != '\0') {
8          printf("texto[%d] = %c\n", i, texto[i]);
9          i++;
10     }
11
12     return 0;
13 }
```

Contando vogais em uma string

```c
#include <stdio.h>

int main() {
    char texto[] = "Programacao";
    int i = 0, vogais = 0;

    while (texto[i] != '\0') {
        char c = texto[i];

        if (c == 'a' || c == 'e' || c == 'i' ||
            c == 'o' || c == 'u' || c == 'A' ||
            c == 'E' || c == 'I' || c == 'O' || c == 'U') {
            vogais++;
        }

        i++;
    }

    printf("Quantidade de vogais: %d\n", vogais);
    return 0;
}
```

Invertendo uma string manualmente

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char texto[] = "Brasil";
6      int i, tam = strlen(texto);
7
8      for (i = tam - 1; i >= 0; i--) {
9          printf("%c", texto[i]);
10     }
11     printf("\n");
12
13     return 0;
14 }
```

Transformando letras minúsculas em maiúsculas

```c
#include <stdio.h>
#include <ctype.h>

int main() {
    char texto[] = "algoritmos";
    int i = 0;

    while (texto[i] != '\0') {
        texto[i] = toupper((unsigned char)texto[i]);
        i++;
    }

    printf("%s\n", texto);
    return 0;
}
```

Erros comuns com strings

- Esquecer espaço para o `\0`.
- Tentar copiar string com `=`.
- Ler mais caracteres do que o vetor suporta.
- Usar `strcat` sem espaço suficiente.
- Comparar strings com `==` em vez de `strcmp()`.

Errado:

```
1 if (a == b) { ... }
```

Certo:

```
1 if (strcmp(a, b) == 0) { ... }
```

Inicialização e lixo de memória

- Ao inicializar uma string na declaração, as posições restantes do vetor podem ficar preenchidas com `\0`.
- Porém, em outras formas de uso, podem existir posições com lixo de memória.

Exemplo:

```
1 char str[6] = "Oi";
```

- Nesse caso, o compilador pode preencher o restante com zeros.
- Já após certas leituras ou cópias, as posições não usadas podem não estar zeradas.

Exemplo de estouro de buffer

```c
#include <stdio.h>

int main() {
    char nome[5];

    // Suponha que o usuario digite "Roberto"
    fgets(nome, sizeof(nome), stdin);

    printf("%s\n", nome);
    return 0;
}
```

- Aqui só cabem 4 caracteres úteis + \0.
- Se a entrada for maior, o texto será truncado.
- Isso é melhor do que ultrapassar o limite do vetor.

Resumo final

- String em C é um vetor de `char` terminado em `\0`.
- Sempre reserve espaço para o terminador.
- Use `printf(%s)` e prefira `fgets()` para leitura.
- A biblioteca `string.h` facilita operações comuns:
  - `strlen`
  - `strcpy`
  - `strcat`
  - `strcmp`
- Tenha cuidado com limites de vetor e estouro de buffer.

Exemplo final integrando vários conceitos

```c
#include <stdio.h>
#include <string.h>
int main() {
    char nome[40];
    char sobrenome[40];
    char completo[90];
    printf("Digite o nome: ");
    fgets(nome, sizeof(nome), stdin);
    nome[strcspn(nome, "\n")] = '\0';
    printf("Digite o sobrenome: ");
    fgets(sobrenome, sizeof(sobrenome), stdin);
    sobrenome[strcspn(sobrenome, "\n")] = '\0';
    strcpy(completo, nome);
    strcat(completo, " ");
    strcat(completo, sobrenome);
    printf("Nome completo: %s\n", completo);
    printf("Quantidade de caracteres: %zu\n", strlen(completo
        ));
    return 0;
}
```

Exercício 1

Leia um nome e exiba informações sobre a string.

Crie um programa em C que:

- leia um nome completo usando `fgets()`;
- remova o caractere `\n`, se existir;
- mostre o nome digitado;
- mostre a quantidade de caracteres usando `strlen()`.

Exemplo de saída esperada:

```
Digite seu nome: Maria Silva
Nome digitado: Maria Silva
Quantidade de caracteres: 11
```

Objetivo: praticar leitura, remoção de `\n` e uso de `strlen()`.

Exercício 2

Compare duas palavras digitadas pelo usuário.

Crie um programa em C que:

- leia duas strings usando `fgets()`;
- remova o `\n` das duas;
- compare as strings usando `strcmp()`;
- informe se elas são iguais ou diferentes.

Exemplo de saída esperada:

```
Digite a primeira palavra: casa
Digite a segunda palavra: casa
As strings sao iguais.
```

Objetivo: praticar leitura de strings e comparação com `strcmp()`.